k-NN, Naive Bayes and Support Vector Machines

Lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · reading time about 100 minutes

Contents

Lesson plan . . . 1
1. Why this lesson exists . . . 2
    1.1 The dataset, and the point it makes immediately . . . 3
2. k-nearest neighbours . . . 4
    2.1 The whole algorithm . . . 4
    2.2 k is the bias-variance dial, made visible . . . 4
    2.3 What it costs . . . 6
3. The curse of dimensionality . . . 6
    3.1 The demonstration . . . 6
    3.2 Why: distances stop varying . . . 6
    3.3 What the curse is, and is not . . . 8
4. Naive Bayes . . . 9
    4.1 Bayes' rule, and where the difficulty is . . . 9
    4.2 The assumption . . . 9
    4.3 On the pumps, the assumption is false and costs almost nothing . . . 13
    4.4 One step away, worse than guessing . . . 16
    4.5 Its probabilities are not probabilities . . . 18
    4.6 When to use it anyway . . . 18
5. Support vector machines . . . 19
    5.1 The margin: a different criterion . . . 19
    5.2 The optimisation, and the soft margin . . . 20
    5.3 On the pumps, the margin alone does not help . . . 22
    5.4 The kernel trick . . . 22
    5.5 Training and inference, step by step . . . 25
    5.6 Gamma, C, and overfitting you can see . . . 26
6. The three compared . . . 28
    How to choose, in practice . . . 28
7. Summary . . . 28
    Homework . . . 29
    Notation used in this lesson . . . 29

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercises 4 and 5 discussed; a problem no line solves	Slides 2–6
0:10–0:30	20	k-nearest neighbours, and choosing k	Slides 7–14
0:30–0:52	22	The curse of dimensionality	Slides 15–21
0:52–1:14	22	**Notebook 01** — k-NN and the curse	Slide 22
1:14–1:26	12	**Break**	Slide 23
1:26–1:50	24	Naive Bayes, and when its assumption matters	Slides 24–43
1:50–2:06	16	**Notebook 02** — where it fails	Slide 44
2:06–2:34	28	Margins, support vectors, the kernel trick	Slides 45–64
2:34–2:50	16	**Notebook 03** — kernels in practice	Slide 65
2:50–3:00	10	The three compared; homework	Slides 66–69
	180	**Total**	**69 slides, 3 notebooks**

---

1. Why this lesson exists

Lessons 3 and 4 built linear models and spent most of their effort on how to fit them honestly. Lesson 5 built the apparatus for telling whether a model works.

None of that told you **which model to reach for**, and this lesson is the first that offers a choice. Three families, each attacking classification from a completely different direction:

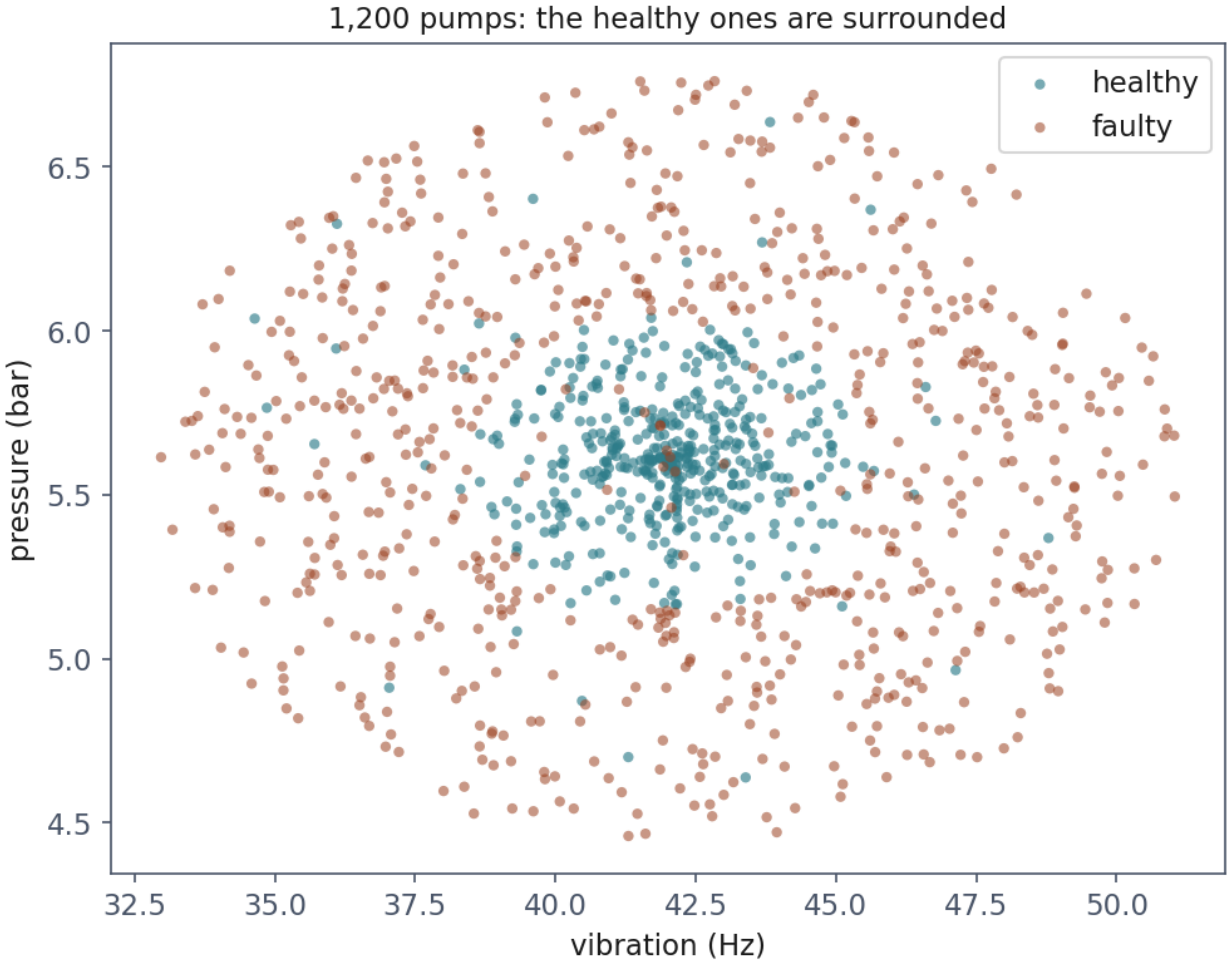
- **k-nearest neighbours** makes no assumptions and does no fitting. It remembers the data and votes.
- **Naive Bayes** makes one very strong assumption and, in exchange, trains in a single pass and works in thousands of dimensions.
- **Support vector machines** pick a boundary by a criterion nothing else in this course uses, and then change coordinates when no boundary exists.

They are not three ways of doing the same thing, and the differences between them are the content of the lesson.

1.1 The dataset, and the point it makes immediately

1,200 industrial pumps, two readings each: vibration in Hz and pressure in bar. A pump has a design operating envelope, and it is **faulty when its readings fall outside that envelope — whether too low or too high**.

That one physical fact decides everything that follows. The healthy pumps form a disc around the design point; the faulty ones form the annulus around them.



1,200 pumps. The healthy ones are surrounded, because a pump can fail by running too slow as well as too fast. No straight line separates these classes.

4% of the labels are deliberately flipped, so no model can exceed roughly 0.96. That ceiling is lesson 5's noise floor arriving in a classification problem, and it is the number to compare every score below against.

Before building anything, it is worth measuring what a straight boundary costs:

Model	Cross-validated accuracy
Always predict the majority class	0.613
Logistic regression (lesson 4)	**0.613 ± 0.000**

Logistic regression has not been narrowly beaten. It has learned nothing at all, and it predicts "faulty" for every pump because with a straight boundary that is genuinely its best available answer. The zero standard deviation across folds is the giveaway: a model that gives everything the same answer is perfectly consistent.

---

2. k-nearest neighbours

2.1 The whole algorithm

> To classify a new point, find the k training points closest to it, and take the majority vote.

There is no training step. "Fitting" means storing the data, which is why k-NN is called a **lazy learner** — an unusually honest name for an algorithm.

Two decisions hide in that sentence, and both matter.

What "closest" means. Almost always Euclidean distance:

$$d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2}$$

Because every feature contributes to that sum through its own units, the features must be on comparable scales. Vibration ranges over tens of Hz and pressure over a couple of bar, so without scaling vibration would decide every neighbour by itself. Lesson 2's scaling argument arrives here with immediate consequences rather than as hygiene.

What k is. Section 2.2.

2.2 k is the bias-variance dial, made visible

Lesson 5 decomposed error into bias and variance. In k-NN you can watch the trade-off directly by turning one integer.

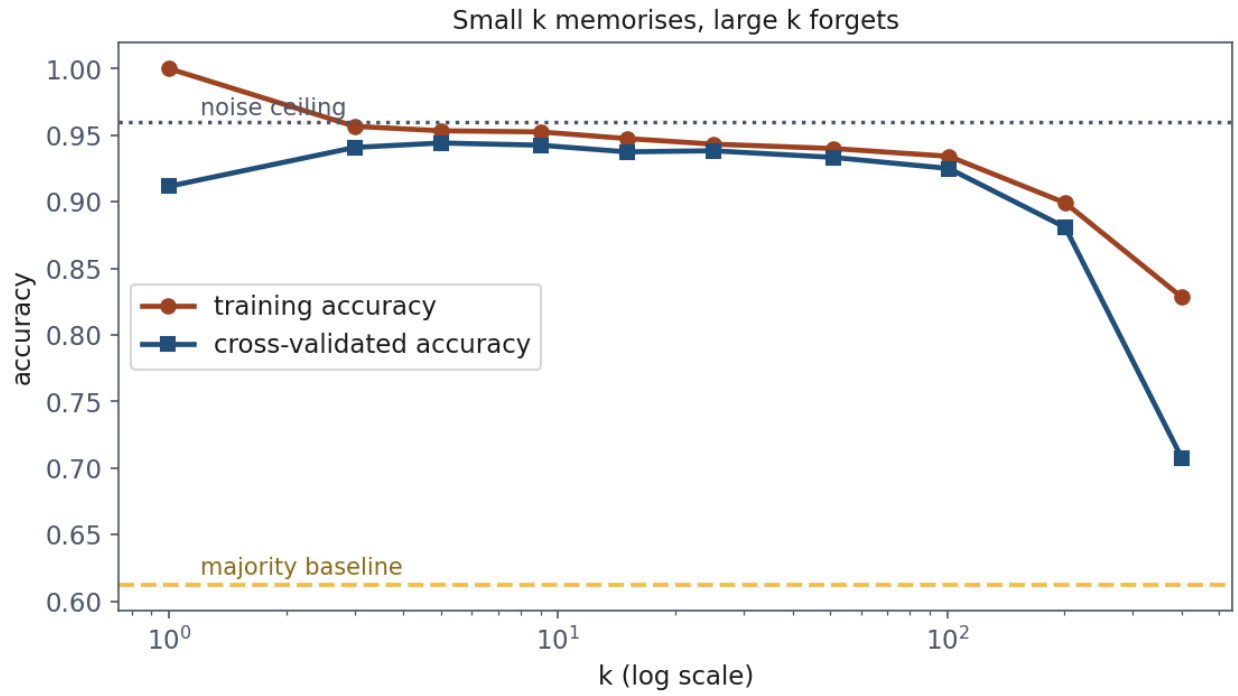
Small k — the boundary follows every point, including mislabelled ones. Low bias, high variance. At $k = 1$ the training accuracy is **exactly 1.000**, always, on any dataset: every point is its own nearest neighbour. That is the purest illustration of lesson 5's point that a training score measures nothing.

Large k — the vote is taken over a wide neighbourhood, so the boundary smooths and eventually stops following real structure. High bias, low variance.

Measured on the pumps:

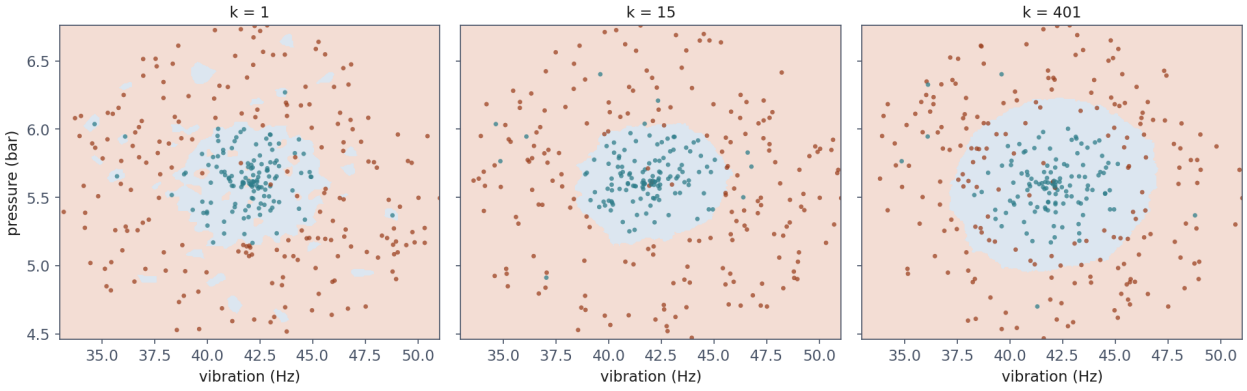
k	Training	Cross-validated
1	**1.000**	0.912
3	0.957	0.941
5	0.953	**0.944**
15	0.948	0.938
51	0.940	0.933

k	Training	Cross-validated
201	0.899	0.881
401	0.828	0.708



The two curves and the two lines that bound them: the noise ceiling at 0.96, which nothing can exceed, and the majority baseline at 0.613, which everything should. Note where the training curve starts.

The best value, 5, sits where the neighbourhood is wide enough to average out the flipped labels and narrow enough to still follow the boundary.



The same data at three values of k. At k = 1 the boundary is ragged, with islands around individual mislabelled points. At k = 15 it is a clean disc, close to the envelope that generated the data. At k = 401 it has ***inflated*** *past the true envelope and swallows faulty pumps — it has not collapsed to one class, it has stopped following the boundary and started averaging over it.*

2.3 What it costs

No training time at all, and you pay at every prediction instead. A naive implementation compares the query against **every** training point:

$$O(mn) \text{ per prediction, for } m \text{ training rows and } n \text{ features}$$

For 1,200 pumps that is nothing. For ten million rows answering a thousand queries a second it is the entire engineering problem, and it is why approximate nearest-neighbour indexes are an industry.

There is a second cost that is easy to miss: **the model is the dataset**. You cannot ship the model without shipping the training data, which is a legal question as much as a practical one — see this lesson's companion reading and lesson 2's.

---

3. The curse of dimensionality

3.1 The demonstration

k-NN just solved a problem that defeated logistic regression completely. Here is the price.

Add columns of **pure noise** — drawn from a normal distribution, unrelated to anything. The two real readings are untouched, so the problem is exactly as solvable as before. Only the number of columns changes.

Noise columns	Total columns	Accuracy	Above baseline
0	2	0.938	+0.325
5	7	0.854	+0.242
10	12	0.762	+0.149
25	27	0.662	+0.049
50	52	0.602	**−0.011**
100	102	0.578	−0.035

The signal never left. Those two columns are still there and still sufficient. But by fifty noise columns, k-NN is below the majority baseline: a model that ignored the data entirely would now do better.

3.2 Why: distances stop varying

The mechanism is geometry, not statistics, and it has nothing to do with k-NN specifically.

The picture first. Scatter points uniformly in a cube, pick one, and measure the distance to its nearest neighbour and to its farthest. In two dimensions those are very different numbers, and that difference is exactly what makes "nearest" a meaningful word.

Now add dimensions, and follow a single distance while it happens. Squared distance is a sum with one term per dimension, and for points scattered independently those terms are independent draws from the same distribution:

$$d(x, x')^2 = \sum_{j=1}^{n} (x_j - x'_j)^2$$

Call the mean of one term μ and its variance v. Both are fixed by how a single coordinate is distributed — for coordinates uniform on $[0, 1]$ they are $\mu = 1/6$ and $v = 7/180$ — and, crucially, **neither depends on n.**

Where the two numbers come from. Both belong to a single pair of coordinates, so one dimension is enough, and the only thing to know is how the gap between two random points on a line is distributed. Write $D = x_j - x'_j$. The mean needs nothing but variances: D has mean zero, so $\mathbb{E}[D^2] = \mathrm{Var}(D)$, and for independent points the variances add. A uniform coordinate on $[0, 1]$ has variance $\mathbb{E}[x^2] - \mathbb{E}[x]^2 = 1/3 - 1/4 = 1/12$, so $\mu = 1/12 + 1/12 = 1/6$. The variance needs the fourth power, $v = \mathbb{E}[D^4] - \mu^2$, and for that the shape of D. A small gap can arise from many positions of the pair, a gap near 1 only from one point at each end, so D has the triangular density $1 - |t|$ on $[-1, 1]$. It is symmetric, so integrate over $[0, 1]$ and double:

$$\mathbb{E}\left[D^4\right] = 2\int_0^1 t^4(1-t)\,dt = 2\left(\tfrac{1}{5} - \tfrac{1}{6}\right) = \tfrac{1}{15}, \qquad v = \tfrac{1}{15} - \tfrac{1}{36} = \tfrac{7}{180} \approx 0.039$$

The same density gives the mean by a second route, $2\int_0^1 t^2(1-t)\,dt = 2(1/3 - 1/4) = 1/6$.

Adding n such terms adds their means, and because they are independent it adds their variances too:

$$\mathbb{E}\left[d^2\right] = n\mu, \qquad \mathrm{Var}\left(d^2\right) = nv, \qquad \text{so} \quad \mathrm{sd}\left(d^2\right) = \sqrt{nv}$$

That is the entire mechanism: the centre grows like n and the spread only like $\sqrt{n}$. What the word "nearest" needs is not a large spread but a large spread *measured against the distances themselves*, and dividing one by the other shows what becomes of it:

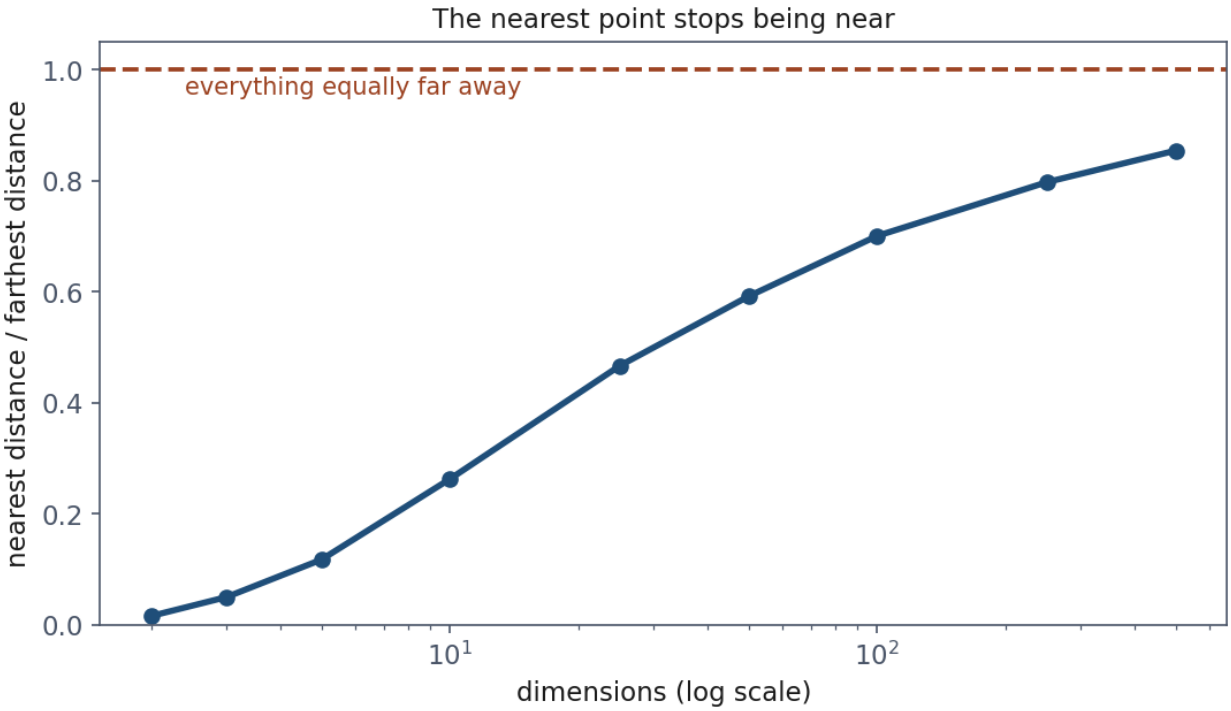
$$\frac{\mathrm{sd}(d^2)}{\mathbb{E}\left[d^2\right]} = \frac{\sqrt{nv}}{n\mu} = \frac{1}{\sqrt{n}} \cdot \frac{\sqrt{v}}{\mu}$$

For the uniform cube $\sqrt{v}/\mu = 1.18$, so the relative spread is $1.18/\sqrt{n}$: **0.84 in two dimensions, 0.12 in one hundred.** Going back from d^2 to d does not change the rate — at $n = 100$ the relative spread of the distance itself is 0.059, half that of its square.

Every pairwise distance is a draw from that distribution, the nearest and the farthest among them included. Squeeze the distribution against its own centre and those two are squeezed together with it, so the ratio of the smaller to the larger is pushed towards 1:

Dimensions	nearest ÷ farthest
2	0.016
10	0.263
50	0.592
100	**0.701**

Dimensions	nearest ÷ farthest
500	0.855



As dimensions grow, the nearest point stops being near. The ratio climbs towards 1, where every point is the same distance from every other.

In two dimensions the nearest point is about 2% as far away as the farthest. "Nearest" is a strong claim.

In one hundred dimensions it is 70% as far away as the farthest. The nearest point is barely nearer than a random one, and a vote among "the five nearest" is close to a vote among five taken at random.

3.3 What the curse is, and is not

It is **not** that high-dimensional problems are inherently unlearnable. Lesson 9's networks work in thousands of dimensions.

It is that **methods built on distance lose their footing**, because the quantity they depend on stops varying. That includes k-NN, k-means (lesson 8), and RBF kernels, which measure similarity through the same Euclidean distance.

The predictable mistake, and why the instinct is sound. Adding features because they might help is good practice with a linear model, where an irrelevant feature costs you a coefficient near zero and very little else. With k-NN it costs you a dimension in the distance, and dimensions are what the method is made of. The same habit, transferred one lesson later, does real damage.

---

4. Naive Bayes

4.1 Bayes' rule, and where the difficulty is

We want $P(y = c \mid x)$. Bayes' rule turns it into quantities we can estimate:

$$P(y = c \mid x) = \frac{P(x \mid y = c)\, P(y = c)}{P(x)}$$

$P(y = c)$ is the prior — a count. $P(x)$ is identical across classes, so it cannot change which class wins and can be dropped.

Dropped for choosing, not lost. $P(x)$ is the probability of seeing this combination of readings whatever the class — by the law of total probability, the sum of the numerators, $\sum_c P(x \mid y = c)\, P(y = c)$. That is why dropping it costs nothing even when a probability is wanted: divide each class's score by the sum of the scores and the denominator is back, exactly. The tempting conclusion is that a model which drops $P(x)$ can no longer report a calibrated probability. It is reasonable, because a term has been thrown away, and wrong, because that term is recoverable from what is kept. The pumps are 465 healthy and 735 faulty out of 1,200, priors 0.3875 and 0.6125. Take illustrative likelihoods of 0.12 and 0.03 for one pump's readings: the scores are 0.0465 and 0.0184, their sum, 0.0649, is $P(x)$, and the probabilities are 0.717 and 0.283. scikit-learn's `predict_proba` does exactly this, in logarithms. Where Naive Bayes does lose calibration is in the numerator, Section 4.5.

Everything hard is in $P(x \mid y = c)$: **the probability of this exact combination of readings among examples of that class.**

With two features that is a two-dimensional density and we could estimate it. With twenty it is a twenty-dimensional one, and no quantity of data populates a twenty-dimensional space — the curse of Section 3, arriving from an entirely different direction.

4.2 The assumption

Given the class, the features are independent of one another.

What it says, concretely. Once you know whether a pump is faulty, reading its vibration tells you nothing more about its pressure. Writing x_1 for the vibration and x_2 for the pressure:

$$P(x_2 \mid x_1, y = c) = P(x_2 \mid y = c)$$

written $x_1 \perp x_2 \mid y = c$. Counts make the difference from plain independence visible. Take, as an illustration, 1,000 pumps, 600 faulty and 400 healthy. Among the faulty, 80% vibrate high and 70% run at low pressure; among the healthy, 10% and 20%; and within each class the two readings are independent:

	High vibration	Low pressure	Both
Faulty (600)	480	420	$600 \times 0.8 \times 0.7 = 336$
Healthy (400)	40	80	$400 \times 0.1 \times 0.2 = 8$
All (1,000)	520	500	344

Inside a class the assumption holds: of the 480 faulty pumps vibrating high, 336 — **70%** — run at low pressure, exactly the rate among all faulty pumps. Across the whole fleet it does not: of the 520 pumps vibrating high, 344 — **66%** — run at low pressure, against **50%** overall. The two readings are correlated because both point towards "faulty", and the class explains all of it.

This is the mistake to expect. "Independent given the class" gets read as "independent", and a student who finds two correlated features concludes that the assumption is broken. It may not be: correlation the class explains is exactly what the assumption allows. The reverse happens too, and the second pair of sensors met in Section 4.4 is that case: there the overall correlation is -0.057, as good as none, while within the healthy pumps it is $\mathbf{+0.821}$ and within the faulty $\mathbf{-0.826}$: once the class is known, one sensor very nearly determines the other, and the assumption fails completely.

If the assumption holds, the joint density factorises:

$$P(x \mid y = c) = \prod_{j=1}^{n} P(x_j \mid y = c)$$

and each factor is a one-dimensional density estimated from the rows of that class. The classifier is then

$$\hat{y} = \arg\max_{c} \; P(y = c) \prod_{j=1}^{n} P(x_j \mid y = c)$$

In practice that product is never computed: hundreds of small probabilities multiplied together underflow to zero in floating point, exactly as lesson 4's log-likelihood did. The repair is available because the logarithm is **increasing** — whichever class makes the product largest makes its logarithm largest too, so swapping one for the other cannot change which class wins — and because the logarithm of a product is a sum of logarithms (natural logarithms throughout, so that e undoes them):

$$\hat{y} = \arg\max_{c} \left[\log P(y = c) + \sum_{j=1}^{n} \log P(x_j \mid y = c) \right]$$

The two expressions pick the same class. The second adds $n + 1$ numbers of ordinary size instead of multiplying $n + 1$ tiny ones.

What training computes: Gaussian Naive Bayes. The factorisation leaves one choice open — what shape each one-dimensional density has. The Gaussian version, the one used throughout this lesson, gives every feature in every class its own bell curve:

$$P(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left[-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \right]$$

where μ_{jc} and σ_{jc}^2 are the mean and the variance of feature j among the training rows of class c. Training is therefore three kinds of average and nothing else: the class priors, then a mean and a variance per feature per class. For n features and K classes that is $K(2n + 1)$ numbers, found in one pass, with no gradient descent and nothing to converge. On the pumps it is ten:

Class	Prior	Vibration: mean	sd	Pressure: mean	sd
Healthy	0.3875	41.986 Hz	**1.695**	5.617 bar	**0.219**
Faulty	0.6125	42.015 Hz	**4.636**	5.616 bar	**0.576**

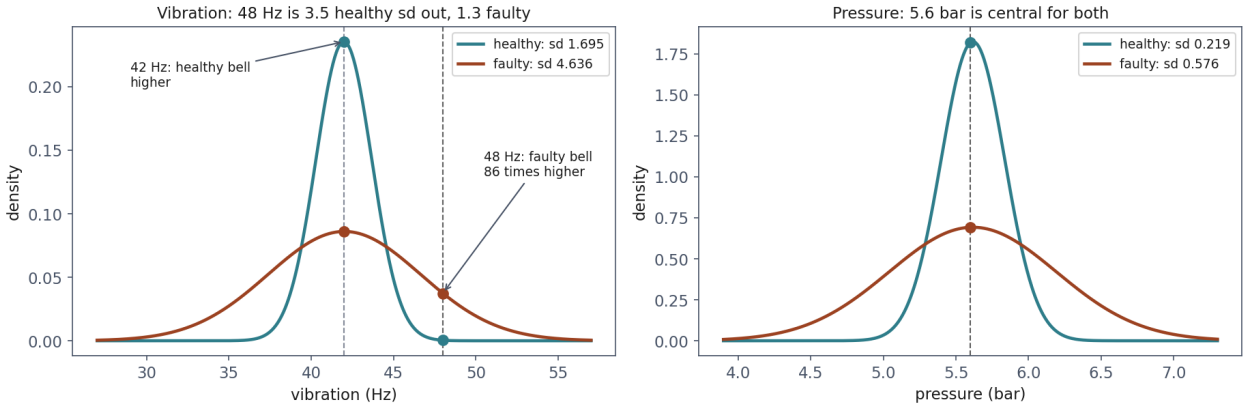
The centres are the same; the widths are not. Healthy pumps are a narrow bell around the design point and faulty ones a wide bell — the disc and the ring, seen one feature at a time. It is also why a linear model will get nowhere on these pumps: it separates classes by placing a boundary between their means, and here there is nothing between the means to find. Naive Bayes compares widths, and Section 4.3 measures what that buys.

Inference for one pump, step by step. Given a new pump's readings $x = (x_1, x_2)$, the model does four things and nothing else:

1. For each class, start from the log prior, $\log P(y = c)$.
2. For each reading, add the log of *that class's* bell at the reading, $\log P(x_j \mid y = c) = -\frac{1}{2}\log(2\pi\sigma_{jc}^2) - (x_j - \mu_{jc})^2/(2\sigma_{jc}^2)$. The second term is a penalty growing with the squared distance from the class's centre, measured in the class's own standard deviations; the first is a charge for a wide bell, which spreads its density thinly.
3. Predict the class with the larger total.
4. If a probability is wanted, normalise as in Section 4.1. With two classes that is $P(\text{faulty} \mid x) = 1/(1 + e^{-\Delta})$, where Δ is the faulty total minus the healthy one.

No training row is looked at: prediction uses the ten stored numbers only, which is the opposite of k-NN.

Here it is with the pumps' numbers. Take a pump at 48 Hz and 5.6 bar, 6 Hz off the design point — about three and a half standard deviations out for a healthy pump, barely more than one for a faulty one. The figure shows what that does to the two bells:



Follow the dashed line at 48 Hz up the left panel: the narrow healthy bell has fallen almost to nothing, the wide faulty one is still well up — 86 times higher. At 42 Hz the narrow bell is on top. On the right, 5.6 bar is central for both classes and barely discriminates; the narrow healthy bell peaks near 1.82, above 1.

The same heights, as logarithms and with the priors added:

Class	log prior	log P(vibration)	log P(pressure)	Total
Healthy	-0.948	-7.745	$+0.598$	-8.095
Faulty	-0.490	-3.286	-0.368	$\mathbf{-4.144}$

Faulty has the larger total, so faulty is the prediction.

From log scores to a probability. Each total is the logarithm of a Bayes numerator, so exponentiating gives the scores back: $e^{-4.144} \approx 0.01585$ for faulty and $e^{-8.095} \approx 0.000305$ for healthy. Normalising as in Section 4.1 divides one by their sum, which is $P(x)$:

$$P(\text{faulty} \mid x) = \frac{e^{T_{\text{faulty}}}}{e^{T_{\text{faulty}}} + e^{T_{\text{healthy}}}} = \frac{0.01585}{0.01616} \approx 0.981$$

and for the other class, from the same denominator, $P(\text{healthy} \mid x) = 0.000305/0.01616 \approx 0.019$ — the two adding to one, as probabilities of the only two classes must.

Divide the top and the bottom by $e^{T_{\text{faulty}}}$ — which leaves the fraction unchanged — and the ratio of the two scores becomes a single exponential, since $e^a/e^b = e^{a-b}$. Only the gap between the two totals survives:

$$P(\text{faulty} \mid x) = \frac{1}{1 + e^{T_{\text{healthy}} - T_{\text{faulty}}}} = \frac{1}{1 + e^{-\Delta}}, \qquad \Delta = T_{\text{faulty}} - T_{\text{healthy}} = -4.144 - (-8.095) = 3.951$$

so $1/(1 + e^{-3.951}) = 1/1.0192 \approx \mathbf{0.981}$, the number `predict_proba` returns. Three things follow. **Only the gap matters**: Δ is the log-odds, and faulty is $e^{3.951} \approx 52$ times as likely as healthy. Read that way the formula is the everyday one for odds: 52 to 1 is a probability of $52/(52+1) = 52/53 \approx 0.981$ for faulty and $1/53 \approx 0.019$ for healthy, and $1/(1 + e^{-\Delta}) = e^{\Delta}/(e^{\Delta} + 1)$ is exactly odds over odds plus one. **The function is lesson 4's sigmoid** (its Section 2.2), fed the difference between two Naive Bayes scores instead of a linear combination of the features. And **the second form is the one to compute**: with hundreds of features the totals can reach -800, where e^{-800} is exactly zero in floating point and the first form reads $0/0$, while Δ stays an ordinary number.

The vibration term decided it. One entry looks wrong and is not: log P(pressure) is *positive*, because a density is not a probability and can exceed 1 — a bell with a standard deviation of 0.219 bar peaks at $1/(0.219\sqrt{2\pi}) \approx 1.82$. The same model puts a pump exactly at the design point, 42 Hz and 5.6 bar, at **0.820** healthy.

Two details of the implementation. scikit-learn adds a tiny amount to every variance, `var_smoothing` — 1.4×10^{-8} here — so that a feature constant within a class cannot produce a division by zero. And scaling makes no difference to this model: standardising a feature moves and stretches both classes' bells by the same amount, which shifts both log scores equally.

Why it is such a good bargain. One n-dimensional estimation problem becomes n one-dimensional ones, each fitted by the averages above. It needs very little data per feature, and adding a feature adds only two numbers per class.

Why it is almost never true. Vibration and pressure are both driven by the operating point. "New" is not independent of "York" given the topic. Symptoms co-occur.

The interesting question is therefore not whether the assumption holds, but **when being wrong about it costs you nothing**.

4.3 On the pumps, the assumption is false and costs almost nothing

Why use a model whose assumption may be false? Because the assumption is not something anyone checks before fitting. Independence given the class is not a precondition to verify first; it is a simplification accepted for what it buys: for each class, one bell per reading — ten numbers in all, priors included — instead of a joint density of vibration and pressure that would need far more data to estimate. Like every modelling choice in this course, it is then judged by the honest score it earns on data it did not see, which is lesson 5's discipline, not by whether the assumption holds. And that score measures something weaker than the assumption. To classify, the model has only to rank the right class first — the sign of the gap between the two log scores — not to get either density right. Domingos and Pazzani (1997) showed that Naive Bayes can be the optimal classifier by that criterion, the one that counts only right and wrong decisions, even where independence is badly violated. This section finds both halves on the pumps: the assumption is false, and the ranking survives it. Section 4.4 is the case where it does not.

Model	Accuracy
Majority baseline	0.613
Logistic regression	0.613
Gaussian Naive Bayes	**0.933**
k-NN, $k = 5$	0.944

A point behind k-NN, and an enormous distance ahead of the linear model. So test the assumption directly — remembering that it concerns independence *given the class*, which is not the same as independence overall:

Correlation between the two readings	
overall	−0.046
within healthy pumps	**−0.006**
within faulty pumps	**−0.049**

What the three rows are. Each is the Pearson correlation between vibration and pressure, computed on a different group: all 1,200 pumps, the 465 healthy ones alone, the 735 faulty ones alone. It runs from −1 to +1; ±1 means the points lie on a straight line, 0 means no straight-line trend at all. The first row is there for contrast. The assumption is about each class separately, so the second and third rows are the ones that test it.

How close to zero is "essentially zero". Even two readings with no relation at all never give exactly 0 on a finite sample: the sample correlation wobbles around zero with a standard deviation of about $1/\sqrt{n}$. That is the yardstick:

Group	n	Expected wobble, $1/\sqrt{n}$	Measured	Measured ÷ wobble
Healthy	465	0.046	−0.006	0.13
Faulty	735	0.037	−0.049	1.3

Both sit well inside the noise a true correlation of zero would produce. In the units that matter: among faulty pumps $r^2 = 0.049^2 \approx 0.0024$, so a straight line through vibration explains about 0.24% of the variation in pressure — nothing. Nor is zero an accident. The generator places pumps at a uniformly random angle around the design point, in a disc for healthy pumps and a ring for faulty ones, so any straight-line tilt in one direction is matched by the opposite tilt; the true within-class correlation is exactly 0, and the measured values are that zero plus sampling noise.

Within each class the readings are essentially uncorrelated — and it is tempting to stop there and declare the assumption true. It is not, because two different claims are in play, and only one of them holds:

Claim, within each class	True here?	What it means
The readings are **uncorrelated**	Yes	no straight line ties vibration to pressure
The readings are **independent**	**No**	knowing the vibration changes what to expect of the pressure

Naive Bayes assumes the second. Independence asks far more than a flat straight line: $P(\text{pressure} \mid \text{vibration}, y = c)$ must equal $P(\text{pressure} \mid y = c)$, the *whole* distribution of pressure unchanged whatever the vibration. Correlation checks only whether its average drifts along a line.

Uncorrelated is not independent. Correlation detects only straight-line dependence, and the dependence here is not a straight line. Healthy pumps fill a disc around the design point and faulty ones the ring around it. On a ring, a pump whose vibration is far from the design point must have a pressure close to it, or it would fall outside the ring. That is a strong dependence, but a symmetric one — far-out vibration goes with central pressure on both sides — so the straight-line summary averages it to zero. Square each reading's standardised distance from the design point, turning "far out" into "large", and it appears:

Correlation of the squared distances	
within healthy pumps	+0.168
within faulty pumps	**−0.420**

Why the square, and not the cube. The transformation has to treat "far out to the left" and "far out to the right" alike, because the dependence is about distance from the centre, not direction. That asks for an *even* function, $f(-z) = f(z)$: the square maps −2 and +2 both to 4. The cube is *odd* — it maps them to −8 and +8, keeps the sign, and so the two far tails go on cancelling exactly as they did for the plain correlation. Measured on the same standardised readings:

Transformation	Healthy	Faulty
z (none)	-0.006	-0.049
z^3 (odd)	-0.058	-0.022
z^2 (even)	$+0.168$	$\mathbf{-0.420}$
$\lvert z\rvert$ (even)	$+0.211$	$\mathbf{-0.454}$
z^4 (even)	$+0.060$	-0.288

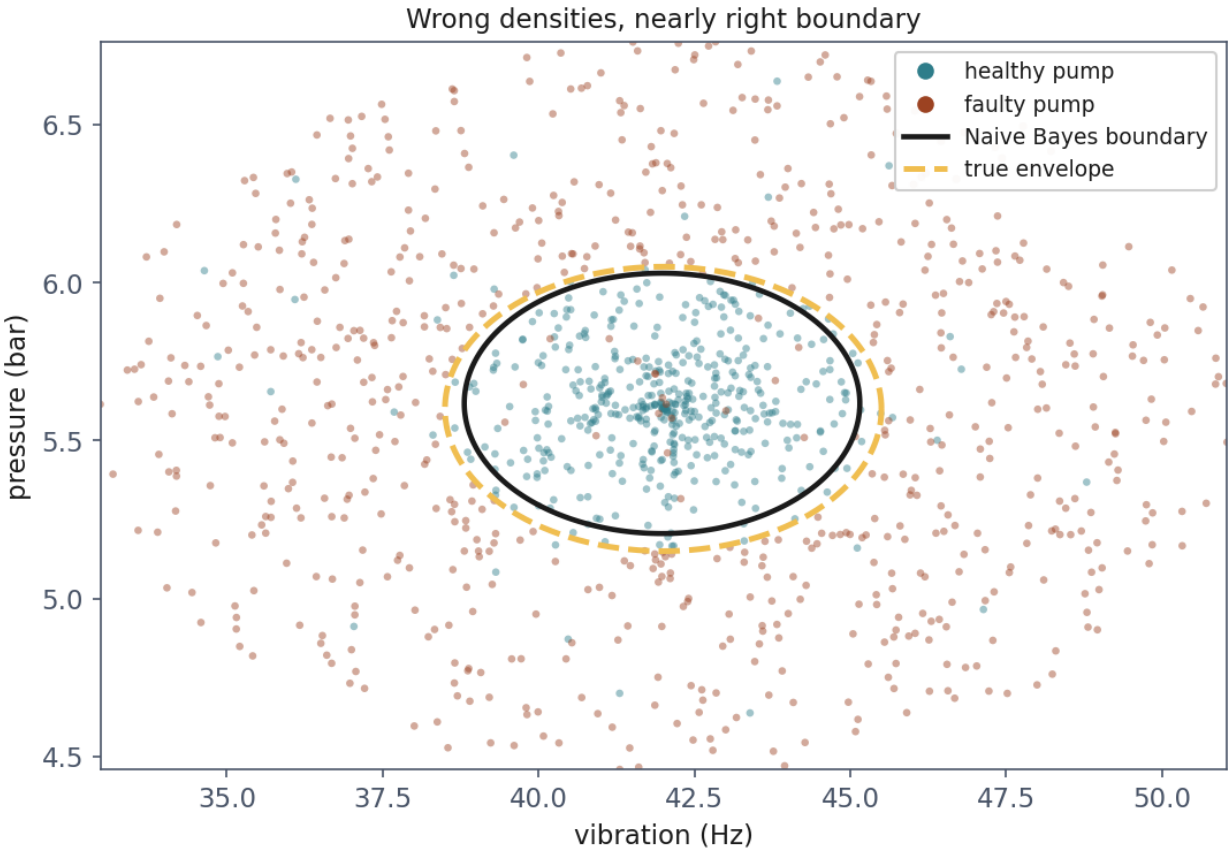
The cube sees nothing, like the raw readings; every even transformation sees the dependence. The absolute value works as well as the square, and the fourth power less well, because it lets a few extreme points dominate. The square is the natural choice among them because it is the geometry the data was built with — the disc and the ring are $z_1^2 + z_2^2$ below or above a radius — and the same coordinate section 5.4's kernel lift will use. The limitation is worth stating: the square worked because we knew the dependence was radial. On real data nobody tells you which transformation to try, which is why a correlation near zero never licenses the word "independent".

Among faulty pumps, when one reading is far out the other is pulled in. The same dependence is plainer as a spread. On standardised readings, the pressure of a faulty pump has a standard deviation of **0.784** when its vibration is extreme — the fifth of faulty pumps furthest out, 147 of them, more than 6.2 Hz from the centre — and **1.309** when it is not: in bar, **0.37** against **0.62**. Its average does not move, which is why the correlation is zero; its spread falls by two fifths. A pump already far out in vibration is in the ring whatever its pressure, so the pressure can sit near the centre; one central in vibration can only reach the ring through a pressure far out. The assumption is false on these pumps. This is the mistake to expect, and it is a reasonable one: a correlation near zero is the check everyone reaches for, and for bell-shaped data it would settle the matter. For a disc and a ring it does not.

Wrong densities, nearly right boundary. So why 0.933? Because of what the model does with its wrong densities. Section 4.2 found the two classes sharing a centre, each reading a narrow bell for healthy pumps and a wide one for faulty. Subtract the healthy log score from the faulty one, using step 2 of the inference recipe with a common centre μ_j. The squared distances survive with the difference of the two widths as their weight, and everything else is a constant:

$$\log \frac{P(\text{faulty})\,P(x \mid \text{faulty})}{P(\text{healthy})\,P(x \mid \text{healthy})} = \sum_j a_j (x_j - \mu_j)^2 + b, \qquad a_j = \frac{1}{2\sigma^2_{j,\text{healthy}}} - \frac{1}{2\sigma^2_{j,\text{faulty}}}$$

with b collecting the log priors and the log widths. Every a_j is positive, because the healthy bell is the narrower, so the faulty score gains on the healthy one as a pump moves away from the centre, in any direction. The boundary is where the two are equal, $\sum_j a_j (x_j - \mu_j)^2 = -b$: an **axis-aligned ellipse** — and the true envelope is exactly that shape. Walking out from the centre until the fitted model's prediction flips puts its ellipse at about **3.17 Hz by 0.41 bar**, against a true envelope of **3.5 by 0.45**: the same proportions, about a tenth too small.

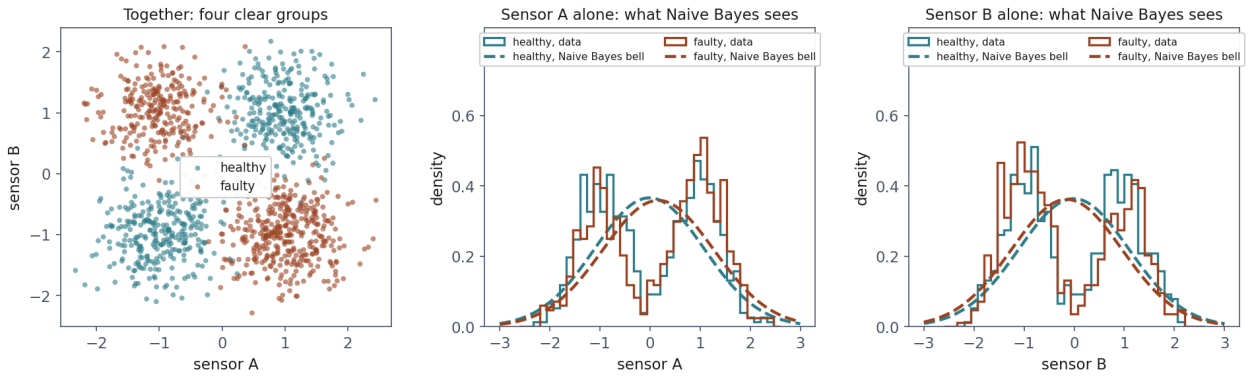


The solid ellipse is where the fitted model changes its mind, the dashed one the envelope the pumps were generated from: same shape and orientation, the model's a little smaller. The densities are wrong; the boundary they imply is nearly right.

The densities Naive Bayes multiplies are wrong; the boundary they imply is nearly right. The cost of the false assumption is the gap from 0.933 to k-NN's 0.944, under a ceiling of 0.96 set by the label noise. This is Section 4.2's closing question answered on data — being wrong about the assumption is cheap when the boundary it implies still has the right shape. Section 4.4 is the case where it does not.

4.4 One step away, worse than guessing

Now a second pair of sensors on the same fleet. The pump is faulty when **exactly one** of the two readings is high: both high is the designed high-load mode, both low is idle, and one without the other is a mismatch between demand and delivery.



Left: together, the two sensors show four clear groups, each class owning two opposite quadrants. Middle and right: what Naive Bayes gets to see — each sensor alone. Solid outlines are the data, two humps each and the same two humps for both classes; dashed curves are the single bell per class that Naive Bayes fits, the two almost on top of each other.

Reading the figure. Teal is healthy and rust is faulty throughout. In the left panel each class owns two opposite quadrants — healthy pumps have both sensors high or both low, faulty ones one high and the other low:

	B high	B low
A high	273 healthy, 9 faulty	10 healthy, 359 faulty
A low	11 healthy, 253 faulty	278 healthy, 7 faulty

The few pumps in the wrong quadrant are the label noise. Now look at sensor A alone, which is what the middle panel does. Among healthy pumps it is high for 273 + 10 and low for 11 + 278 — about half each. Among faulty pumps it is high for 9 + 359 and low for 253 + 7 — about half again, if less evenly. So both outlines have the same two humps, at −1 and +1, and the same is true of sensor B. **Each sensor alone says almost nothing about the class; everything is in the combination.**

The dashed curves are what Naive Bayes makes of that. It fits **one** bell per sensor per class, and a single bell cannot follow two humps, so each sits in the valley between them:

Bell fitted on sensor A	Centre	sd
Healthy	−0.012	1.090
Faulty	+0.167	1.112

Set this beside the pumps of Section 4.2. There the two classes' bells also shared a centre, but their widths differed almost threefold — 1.695 against 4.636 Hz — and the widths carried the class. Here centre and width both coincide, and nothing carries it. The one difference left, the 0.17 between the centres, comes from the sample holding more faulty pumps with A high (359) than with B high (253).

Model	Accuracy
Majority baseline	0.523
Gaussian Naive Bayes	**0.404**
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, $k = 5$	0.967
SVM, RBF kernel	0.972

0.404 — below chance, and well below the majority baseline.

Being below chance looks impossible, and the explanation is the 0.17 just found. The class means on each sensor differ by about that much against a spread near 1, purely as an artefact of a finite sample. That accident is the *only* per-feature evidence available, Naive Bayes has nothing else to multiply, and in this sample it points the wrong way.

A model with no signal does not sit politely at 50%. It follows whatever spurious structure it can find.

4.5 Its probabilities are not probabilities

Measured on the interacting data: mean confidence when correct **0.567**, mean confidence when wrong **0.555**. It cannot tell the difference.

The more common complaint runs the other way, and it is not measured on either dataset here — it needs features that carry the same evidence about the class, near-copies of one another, which neither pair of sensors is. When features do repeat each other, multiplying their probabilities counts the same evidence repeatedly, and Naive Bayes becomes wildly overconfident. Ten near-copies of one informative column is enough: accuracy around 0.75, and two thirds of the predictions asserted above 0.999. Try it — it takes four lines.

Either way, lesson 5's distinction applies: **the ranking may be useful while the probabilities are not.** Naive Bayes is the classic example of that gap, and the reason it should not be used where a calibrated probability is needed — such as the cost calculation of lesson 4, Section 7.2.

4.6 When to use it anyway

Very high dimensions with little data. Text classification is canonical: tens of thousands of word-count features, an assumption that is transparently false, and a method that works anyway — because being roughly right in ten thousand dimensions beats being unable to estimate anything at all.

As a baseline. It trains in one pass. If your tuned model does not beat Naive Bayes, you have learned something quickly and cheaply.

Not when the signal is an interaction. No quantity of data repairs Section 4.4, because the model cannot represent what you are asking of it.

---

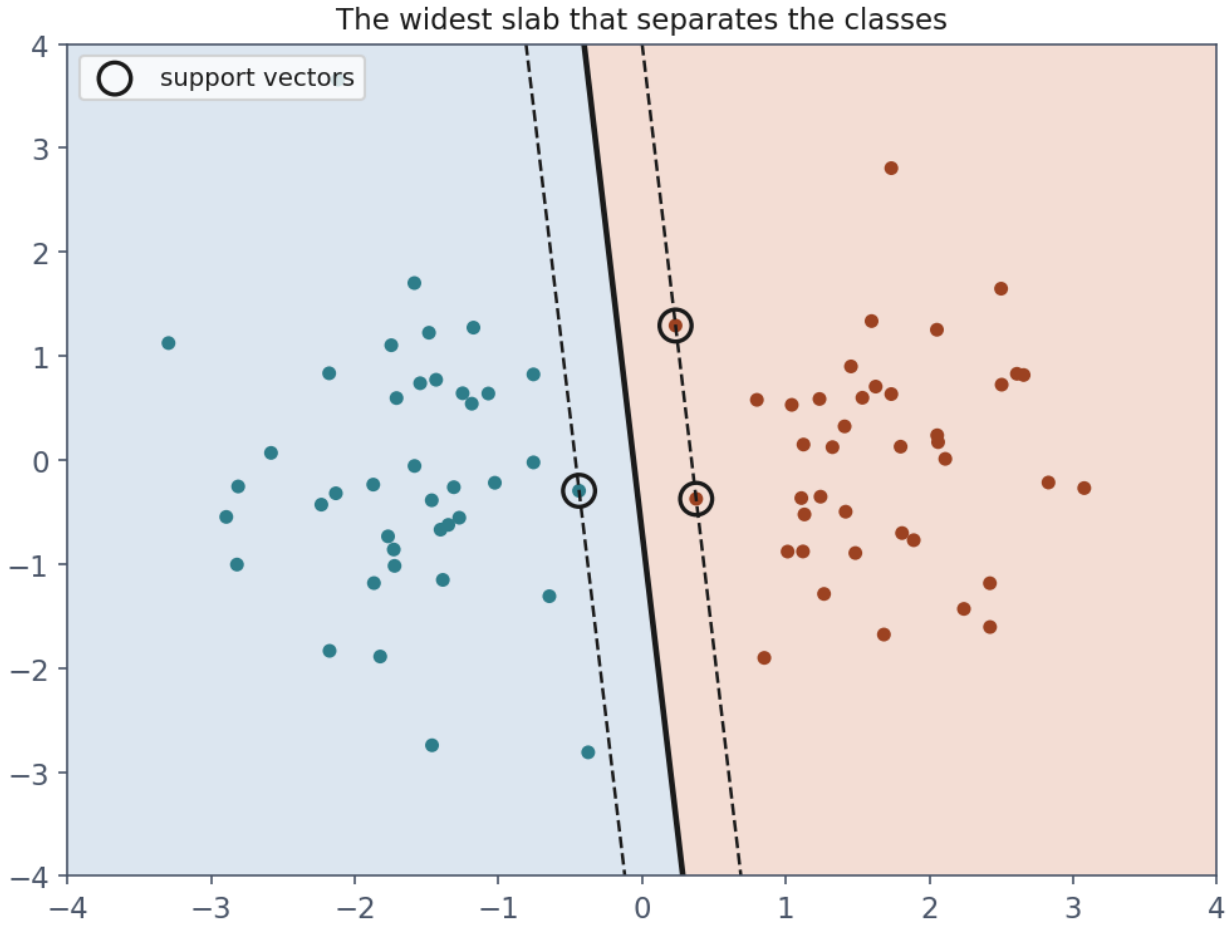
5. Support vector machines

5.1 The margin: a different criterion

On separable data there are infinitely many separating lines, and **every one of them has zero training error**. Minimising the error therefore cannot choose between them; logistic regression breaks the tie with log loss, which is one answer among several.

The support vector machine uses a different one: choose the boundary with the **widest margin**. Push a slab out from the boundary until it touches the nearest point of each class, and pick the boundary that makes the slab thickest.

The intuition. A boundary passing close to a training point is one small perturbation away from getting it wrong. Maximising the distance to the closest points chooses the boundary that tolerates the most movement in the data before it changes its mind. That is a statement about generalisation, not about fit.



The solid line is the boundary, the dashed lines the edges of the slab, and the circled points the support vectors touching it. Of eighty points, three determine the answer; move any of the others and nothing changes.

5.2 The optimisation, and the soft margin

Write the boundary as the set of points where $w^\top x + b = 0$, and the labels as $y_i \in \{-1, +1\}$ rather than $\{0, 1\}$ — a relabelling with no content, chosen because it lets "correctly classified" be written as a single product being positive: $y_i(w^\top x_i + b) > 0$ says the sign of the prediction and the sign of the label agree.

Turning "widest slab" into something a computer can minimise takes three steps.

Step 1 — how far a point is from the boundary. First, w is perpendicular to the boundary: if x and x' both lie on it then subtracting the two equations gives $w^\top(x - x') = 0$, so w is orthogonal to every direction that runs along the boundary. To measure how far a point sits from it, then, project onto the unit vector $w/\|w\|$ pointing across it:

$$\text{distance}(x) = \frac{|w^\top x + b|}{\|w\|}$$

The numerator by itself is not a distance. Multiply w and b both by ten and it grows tenfold while the boundary — the place where the expression is zero — has not moved a millimetre. Dividing by $\|w\|$ is what removes that freedom.

Step 2 — spend the freedom on purpose. That same freedom is a nuisance in an optimisation: infinitely many pairs (w, b) describe one boundary, so the minimum would not be unique. Fix it by a choice — rescale (w, b) so that the training points closest to the boundary satisfy $|w^\top x_i + b| = 1$ exactly. Any separating hyperplane can be written this way, so nothing is given up, and two things are bought:

- **The constraints become clean.** Every point correctly outside the slab now reads $y_i(w^\top x_i + b) \geq 1$, where the 1 means "at least as far out as the closest points are".
- **The margin becomes a formula.** Substituting $|w^\top x_i + b| = 1$ into Step 1, the closest points sit at distance $1/\|w\|$, and the slab reaches that far on both sides:

$$\text{margin} = \frac{2}{\|w\|}$$

That is where the $2/\|w\|$ comes from, and it is worth noticing what has happened: with the scale pinned down, the width of the slab is decided by $\|w\|$ and by nothing else. A short w is a wide margin.

Step 3 — maximise by minimising. Maximising $2/\|w\|$ is minimising $\|w\|$, and minimising $\|w\|$ is minimising $\frac{1}{2}\|w\|^2$ — the same minimiser, because squaring is increasing on non-negative numbers, with the square and the $\frac{1}{2}$ there only so that the derivative comes out as w. So:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i\,(w^\top x_i + b) \geq 1 \ \ \forall i$$

A convex objective with linear constraints, which is the easiest kind of constrained problem there is: one minimum, and no local ones to get stuck in.

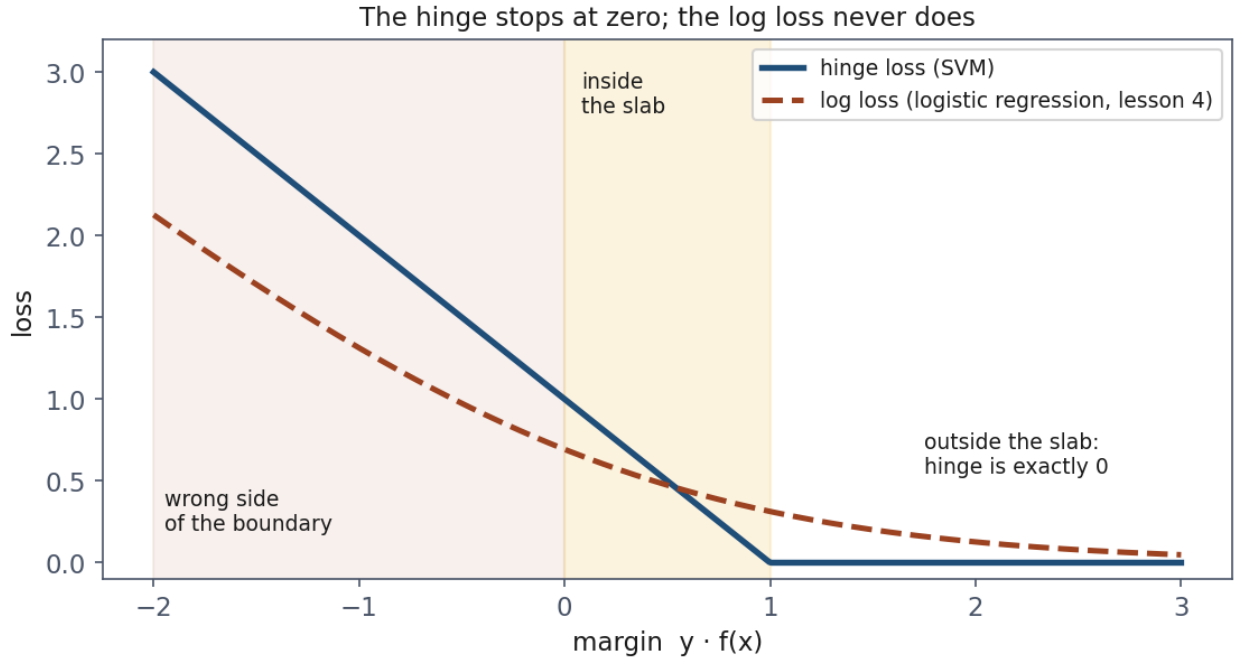
Real data is not separable — ours has 4% of its labels flipped — and this problem then has no solution at all. The fix is to allow violations ξ_i and charge for them:

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\xi_i \quad \text{subject to} \quad y_i\,(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

What ξ_i measures. A point outside the slab on its own side has $\xi_i = 0$ and costs nothing. A point inside the slab but still on the right side of the boundary has $0 < \xi_i < 1$. A point on the wrong side of the boundary has $\xi_i > 1$. At the optimum each ξ_i is as small as its constraint allows, which is

$$\xi_i = \max\left(0,\ 1 - y_i\,(w^\top x_i + b)\right)$$

the **hinge loss**: zero for any point classified with room to spare, then rising in a straight line as the point strays. That zero is the difference from lesson 4's log loss, which shrinks for a well-classified point but never reaches zero, so every row keeps pulling on the answer. Under the hinge, points that are safely right stop mattering altogether — the fact Section 5.4 turns into the method's defining property.



Loss against the margin $y \cdot f(x)$. Right of 1 — outside the slab on its own side — the hinge is exactly zero, while the log loss keeps a small charge: 0.127 at a margin of 2, 0.049 at 3. That zero is where support vectors come from.

C is the price of a training error. Large C makes violations expensive, so the model accepts a narrower margin to classify more training points: low bias, high variance. With a linear kernel the slab stays straight - a larger C narrows it, shifts it or rotates it, and never bends it; with the RBF kernel the boundary can also bend around individual points (Section 5.6). Small C buys a wider, calmer boundary at the cost of some errors.

It is the same dial as k in Section 2 and λ in lesson 3, in a third costume — and note the direction, which catches people out: **large C means less regularisation.**

5.3 On the pumps, the margin alone does not help

Model	Accuracy	Support vectors
SVM, linear kernel	**0.613 ± 0.000**	947 of 1,200 (**79%**)
SVM, RBF kernel	0.947 ± 0.005	278 of 1,200 (23%)

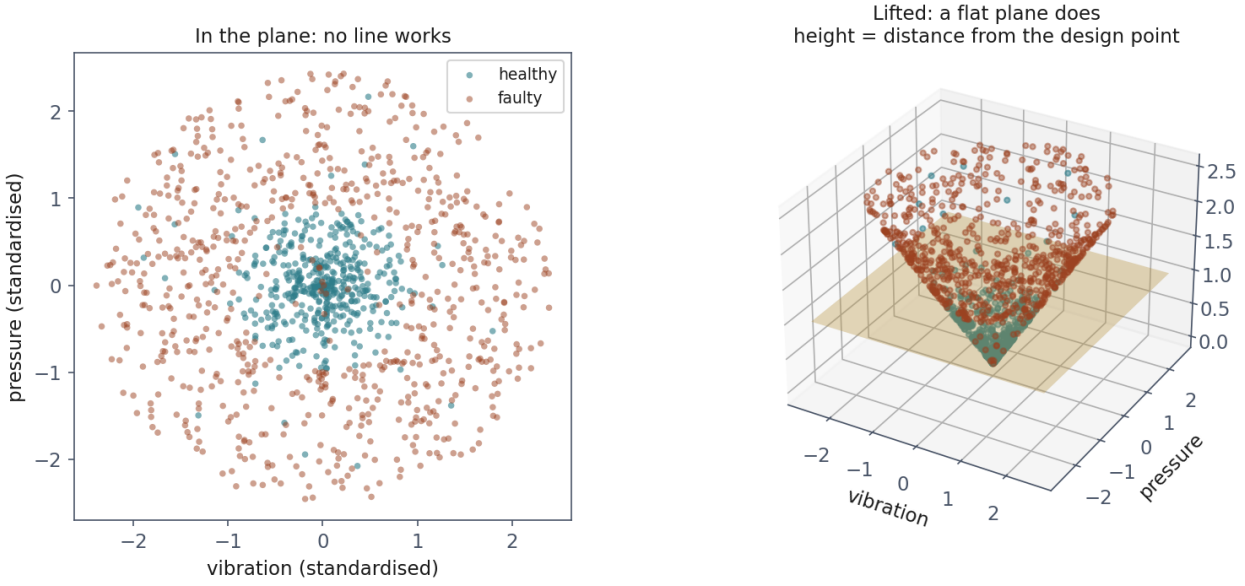
The linear kernel scores the base rate, exactly as logistic regression did. Choosing the best straight line does not help when no straight line works.

The support-vector counts say the same thing in another language. The linear model needs 79% of the training set, because almost every point sits on or inside the margin — there is no slab that separates anything. **A high support-vector fraction is a free warning** that the model is struggling to find room.

5.4 The kernel trick

The picture first. Our healthy pumps sit in a disc, the faulty ones around it, and no line separates them *in the plane*. Add a third coordinate — the distance from the centre — and lift each point to that height. Healthy pumps rise a little, faulty ones a lot, and a flat horizontal plane separates them perfectly.

The classes were always separable. They needed different coordinates.



The same 1,200 pumps twice: in the plane where they were measured, and lifted by their distance from the design point. The gold plane does what no line could.

That is the idea in general: map into a space $\phi(x)$ where a linear boundary works, and run the linear method there. The obstacle is that useful spaces are enormous, sometimes infinite-dimensional, and computing $\phi(x)$ would be impossible.

The trick is that the SVM never needs $\phi(x)$. Three facts carry it. Each is stated first in words, then shown.

Fact 1 — the answer is a weighted vote of the training points. The best w is not some new vector the optimisation invents: it is built out of the training points themselves, each with a weight $a_i \geq 0$ and the sign of its label,

$$w = \sum_{i=1}^{m} a_i y_i x_i$$

The picture: the boundary is held in place by the points that press against it, like a tent held by its pegs, and w records how hard each one presses.

Where it comes from. A constrained minimisation is attacked by attaching a multiplier to each constraint, one price per training point, charged when that point pushes against its own constraint. Writing a_i for the multiplier on point i — most texts write α_i, but this course reserves α for the learning rate — Section 5.2's problem becomes

$$\mathcal{L}(w, b, a) = \tfrac{1}{2}\|w\|^2 - \sum_{i=1}^{m} a_i \left[y_i \left(w^\top x_i + b \right) - 1 \right]$$

to be minimised over w and b and maximised over the $a_i \geq 0$. Setting the two derivatives to zero:

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_i a_i y_i x_i = 0 \quad \Longrightarrow \quad w = \sum_i a_i y_i x_i, \qquad \frac{\partial \mathcal{L}}{\partial b} = -\sum_i a_i y_i = 0$$

The first is Fact 1. Substitute it back into $\mathcal{L}$, using $\sum_i a_i y_i = 0$ to delete the b term, and what is left is a problem in the weights alone — the **dual**:

$$\max_a \ \sum_i a_i - \tfrac{1}{2} \sum_i \sum_j a_i a_j y_i y_j \langle x_i, x_j \rangle \quad \text{subject to} \quad 0 \leq a_i \leq C, \ \ \sum_i a_i y_i = 0$$

The ceiling $a_i \leq C$ is what the soft margin adds: no single point can press harder than the price of a violation, which is the sense in which C really is the price of a point.

Fact 2 — most of the weights are zero. A multiplier is a price paid only where the constraint binds, so the weights come in three kinds, matching the three cases of ξ_i:

Where the point sits	ξ_i	Weight a_i
Outside the slab, on its own side	0	0 — it drops out
Exactly on the edge of the slab	0	between 0 and C
Inside the slab, or on the wrong side	> 0	C, the ceiling

The points with non-zero weight are the **support vectors**, and they are the whole model: the rest vanish from every sum above. That is why, in Section 5.1's figure, three points of eighty decide the answer and the other seventy-seven can be moved freely.

Fact 3 — the data only ever appears as inner products. Look at the dual: the training points enter only through $\langle x_i, x_j \rangle$. And prediction, with Fact 1 substituted into $w^\top x + b$, is

$$w^\top x + b = \sum_i a_i y_i \langle x_i, x \rangle + b$$

— inner products again. Neither fitting nor predicting ever touches a coordinate except inside $\langle \cdot, \cdot \rangle$. Replace each of those with a function that returns the inner product *in the new space* while computing only with the original coordinates,

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

and the whole method runs in ϕ's space without ever visiting it. That function is a **kernel**.

A kernel, checked by hand. Take the lift that lists every product of two coordinates, $\phi(x) = (x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2)$, and two points $a = (1, 2)$ and $b = (3, 1)$. Lifting both and multiplying: $\phi(a) = (1,\ 2\sqrt{2},\ 4)$, $\phi(b) = (9,\ 3\sqrt{2},\ 1)$, and $\phi(a) \cdot \phi(b) = 9 + 12 + 4 = 25$. Never lifting at all: $a \cdot b = 5$, and $5^2 = 25$. The same number, and the second route touched only the original coordinates — so $K(a, b) = (a \cdot b)^2$ *is* the inner product in the three-dimensional space, computed without going there.

Notice what that space contains: x_1^2 and x_2^2, whose sum is the squared distance from the centre — the very coordinate the figure above chose by hand. A plane in this space draws a circle or an ellipse in the original one. The kernel found the lift we had to guess.

The kernel scikit-learn uses by default is the **radial basis function**:

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

Read it as a **similarity**: 1 for two identical points, falling towards 0 as they move apart. It halves at a distance of $\sqrt{\ln 2/\gamma}$; for the default on the standardised pumps, $\gamma = 0.5$, that is about 1.2 standard deviations. Every quantity the SVM computes is now a sum of similarities to training points.

Why the RBF space is infinite-dimensional — a paragraph that can be skipped on a first reading. Expand the squared norm as $\|x\|^2 - 2\langle x, x' \rangle + \|x'\|^2$ and the kernel splits into a factor for each point and one joint factor:

$$K(x, x') = e^{-\gamma \|x\|^2}\, e^{-\gamma \|x'\|^2}\, e^{2\gamma \langle x, x' \rangle}$$

and the joint factor is an exponential, whose power series never terminates:

$$e^{2\gamma \langle x, x' \rangle} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} \langle x, x' \rangle^k$$

Each term is a kernel of the kind just checked by hand — $\langle x, x' \rangle^2$ was the case $k = 2$ — so the sum carries products of the coordinates of *every* degree, and $\phi(x)$ has infinitely many entries. One exponential per pair buys all of them.

We chose the lift in the picture above by knowing the answer; the RBF kernel does something equivalent without being told, which is why it works where nobody could guess the right coordinates.

5.5 Training and inference, step by step

Training means finding the weights a_i of Fact 1 by solving the dual, with the kernel substituted for the inner product:

$$\max_a \sum_i a_i - \tfrac{1}{2}\sum_i \sum_j a_i a_j y_i y_j \, K(x_i, x_j) \quad \text{subject to} \quad 0 \le a_i \le C, \quad \sum_i a_i y_i = 0$$

Both i and j range over the m **training** points, so this needs only $K(x_i, x_j)$ — the $m \times m$ kernel matrix, one entry per pair of pumps already seen. There is no new point anywhere in this expression: training never touches the pump it will later be asked to classify. The solver behind scikit-learn's `SVC` starts with every weight at zero, repeatedly picks a pair of weights that can still improve the objective — a pair, because $\sum_i a_i y_i = 0$ means one cannot move alone — sets that pair to its best values with all the others held fixed, and stops when no pair can improve. There is no learning rate and no epoch, and because the problem is convex the answer is the same on every run. The price is the kernel values between pairs of training points, which is why the method slows sharply as the dataset grows past tens of thousands of rows.

What comes out, for the RBF model on the standardised pumps ($\gamma = 0.5$, $C = 1$):

Weight	Pumps	Meaning
$a_i = 0$	922	outside the slab: forgotten
$0 < a_i < C$	10	exactly on the edge of the slab
$a_i = C$	268	inside the slab or on the wrong side

plus an intercept $b = +1.105$. The model is the 278 support vectors, their weights and b; the other 922 pumps can be deleted. And with $C = 1$, 268 of the 278 weights are exactly 1: almost every support vector simply votes with its label.

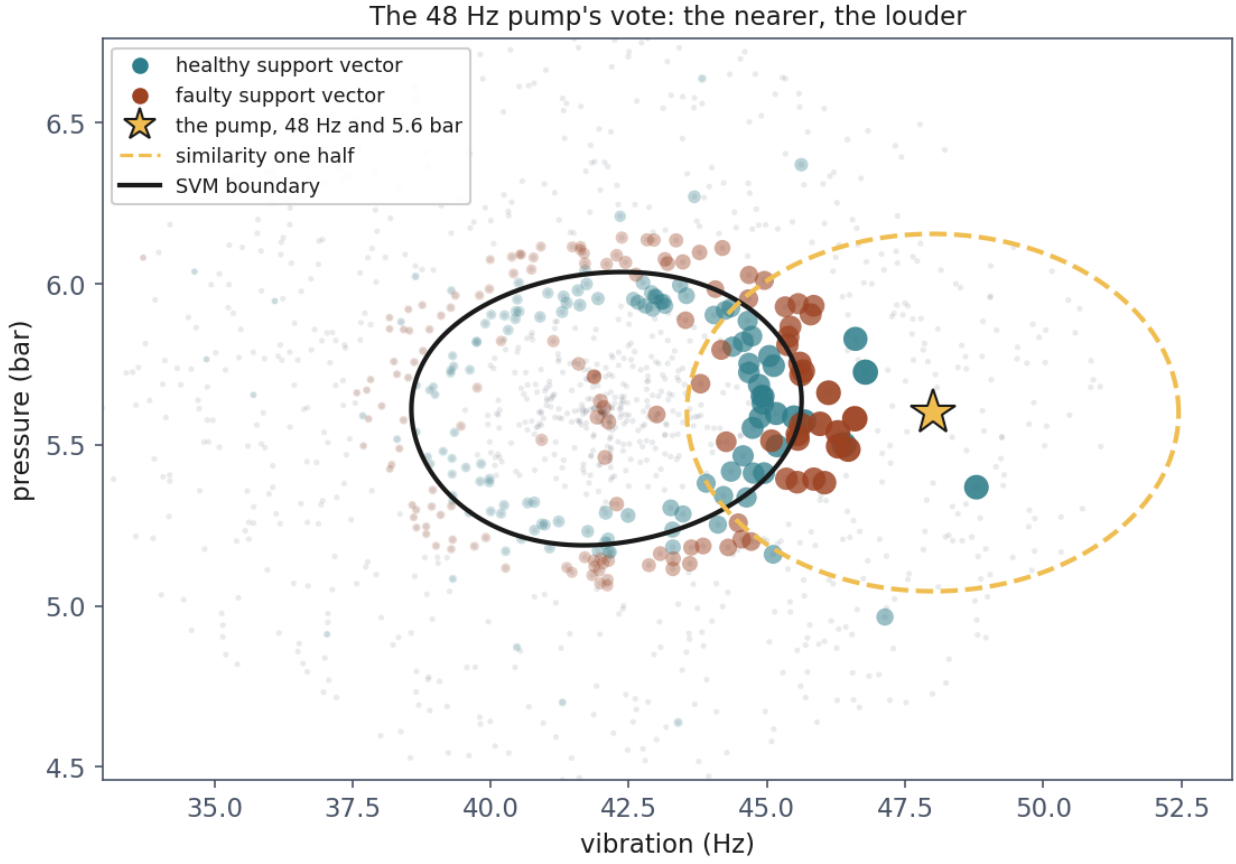
Inference is where the new point finally appears, in the other formula from Fact 3: $\sum_i a_i y_i \, K(x_i, x) + b$, now summed only over the 278 support vectors and with x the pump to classify. Training built the kernel matrix $K(x_i, x_j)$ among pumps it already knew; inference evaluates $K(x_i, x)$ between each surviving support vector and the one pump it has never seen. Same function, two different second arguments — that is the whole distinction. In four steps:

1. Standardise the new pump with the scaler fitted on the training data.
2. Compute its RBF similarity to each of the 278 support vectors.
3. Add up weight × label × similarity over the support vectors, with labels $+1$ for faulty and -1 for healthy, then add b.
4. The sign is the class: positive faulty, negative healthy.

The pump from Section 4.2, at 48 Hz and 5.6 bar:

Term	Value
votes from faulty support vectors	$+37.455$
votes from healthy support vectors	-36.677
intercept b	$+1.105$
total	**$+1.883$** — faulty

The total is exactly what `decision_function` returns. The 56 support vectors with a similarity above one half — those within about 1.2 standard deviations — contribute a net **+1.141** towards faulty, the other 222 only −0.363 between them: **the neighbourhood decides.** Not a single neighbour, though. The five nearest support vectors disagree — pumps just outside the envelope are faulty and some just inside carry flipped labels — so the closest votes nearly cancel, and it is the balance across the neighbourhood, plus the intercept, that sets the sign. Naive Bayes, by a completely different route, called the same pump faulty.

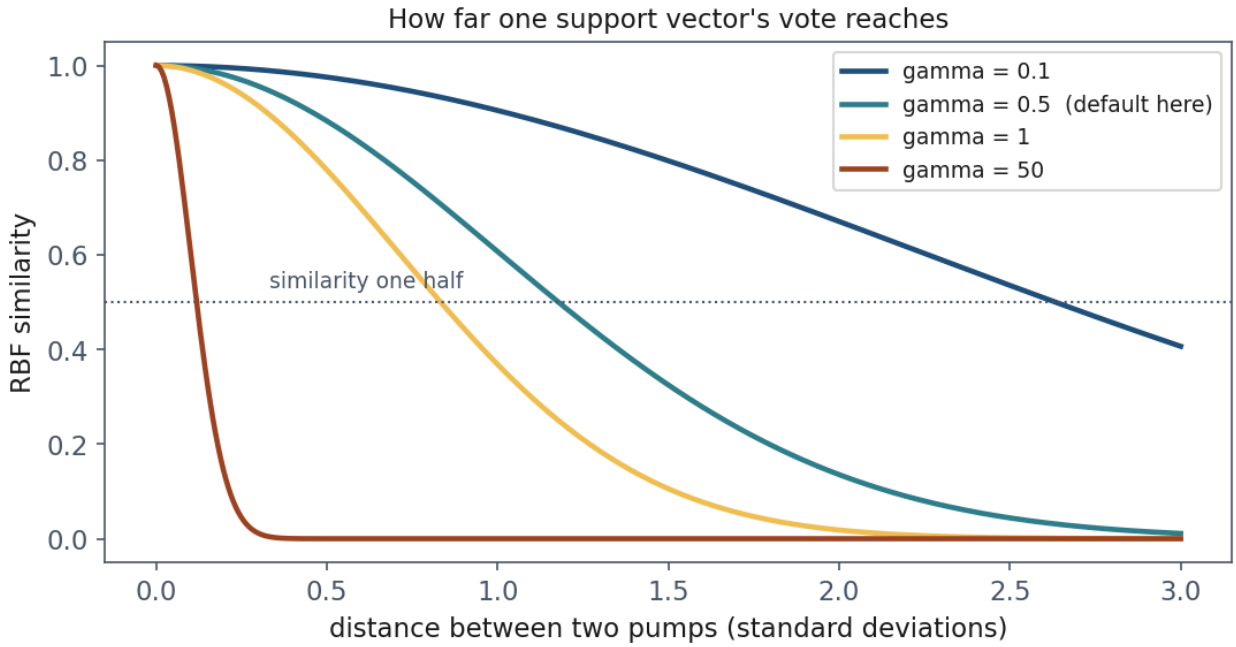


Each support vector sized and shaded by its similarity to the pump (the star). The 56 inside the dashed half-similarity ellipse carry most of the weight; teal and rust mix right beside the star, which is why the nearest votes nearly cancel and the neighbourhood's balance decides.

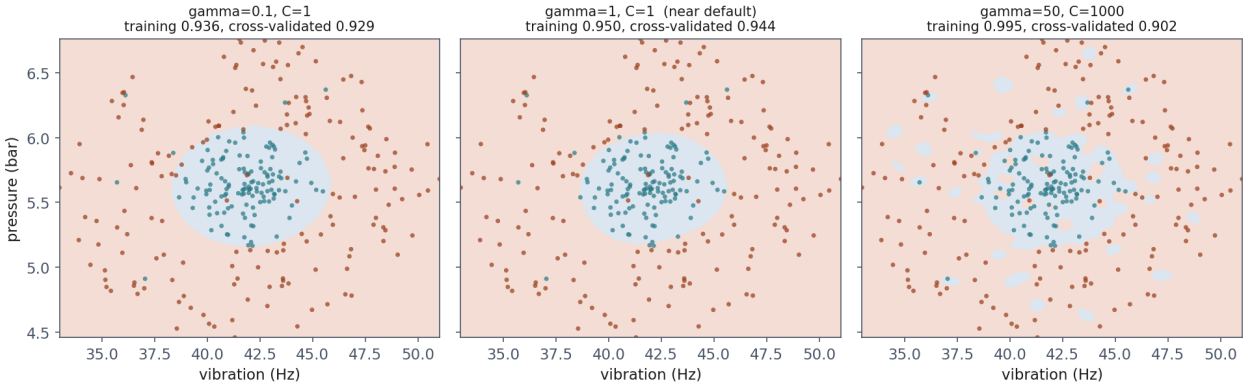
This is k-nearest neighbours again, with three differences: a smooth similarity in place of a hard cut-off at k, only the support vectors voting, and their weights learned. The cost follows: 278 similarities per prediction against k-NN's 1,200 distances. And the output is a score whose sign is the class, not a probability.

5.6 Gamma, C, and overfitting you can see

γ sets how far a single training point's influence reaches — Section 5.4's half-similarity distance, $\sqrt{\ln 2/\gamma}$. At $\gamma = 0.1$ a support vector's vote halves over 2.63 standard deviations, at $\gamma = 1$ over 0.83, and at $\gamma = 50$ over 0.12: small γ gives a wide reach and a smooth boundary, large γ lets each point speak only for its immediate neighbourhood, and the boundary can dissolve into islands.



RBF similarity against distance, one curve per γ. Where each crosses the dotted line at one half is its reach: 2.63, 1.18, 0.83 and 0.12 standard deviations.



Three settings, with the training and cross-validated scores in each title. Read them together, as lesson 5 taught.

Setting	Training	Cross-validated
$\gamma = 0.1$, $C = 1$	0.936	0.929
$\gamma = 1$, $C = 1$	0.950	**0.944**
$\gamma = 50$, $C = 1000$	**0.995**	0.902

The last row has the **highest training score and the lowest honest one** — the signature of overfitting, and here it is visible as well as measurable: the boundary has broken into bubbles around individual points, including the mislabelled ones.

Choosing between these three on the training score would select the worst model with complete confidence.

6. The three compared

Everything in this lesson on the same 1,200 pumps, cross-validated:

Model	Accuracy
Majority baseline	0.613
Logistic regression (lesson 4)	0.613
SVM, linear kernel	0.613
Gaussian Naive Bayes	0.933
k-NN, $k = 5$	0.944
SVM, RBF kernel	**0.947**
noise ceiling	*0.96*

Three methods reach the ceiling by three unrelated routes — remembering the neighbourhood, assuming independence, bending the space — and two do not, both of them linear.

The gap between best and worst is 0.334, larger than any difference this course has shown between a good model and a tuned one. That is the lesson of the table: **choosing the right family matters far more than tuning the wrong one**, and the way to tell which family you need is to look at the data first, as lesson 2 insisted.

How to choose, in practice

Situation	Reach for
Few features, plenty of data, odd-shaped boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
Need a calibrated probability	Not Naive Bayes
Need the model to be small, or fast at prediction	SVM, not k-NN
Signal is an interaction between features	k-NN or a kernel; not Naive Bayes
Need to explain the decision to the person affected	Neither — lesson 3's Resources

7. Summary

- **No straight line separates the pumps**, and both linear models score exactly the base rate, 0.613, with zero variance across folds.
- **k-NN learns nothing and scores 0.944.** Scale first; distance is all it has.
- k **is the bias-variance dial**: training accuracy is exactly 1.000 at $k = 1$ and means nothing.
- **The curse of dimensionality is about distance, not difficulty.** Fifty noise columns took k-NN below the baseline. In 100 dimensions the nearest point is **70%** as far away as the farthest.

- **Naive Bayes assumes independence given the class.** On the pumps it is false — uncorrelated (-0.006) but not independent (-0.420 on the squares) — and costs almost nothing, because the boundary it implies is the right ellipse: 0.933. Where the signal is an interaction it scores **0.404**, below chance.
- **Its probabilities are not calibrated**, in either direction.
- **The margin is a criterion distinct from the error**, and support vectors are the model. A high support-vector fraction is a free warning.
- **The kernel trick separates by changing coordinates**, through inner products in a space never computed: 25 both ways on a two-point example.
- **An SVM is trained as one weight per point**, with no learning rate, and predicts by a similarity-weighted vote of its support vectors — k-NN with learned weights: 278 voters, $+1.883$ for the 48 Hz pump.
- **Choosing the family beats tuning the wrong one**: 0.613 to 0.947 on identical data.

Homework

`Exercises/06_knn_naive_bayes_svm.md`, discussed at the start of Lesson 7, **Friday 20 November 2026**.

Notation used in this lesson

Symbol	Meaning
x, x'	two feature vectors
$d(x, x')$	Euclidean distance between them
k	number of neighbours voting
m, n	number of examples, number of features
μ, v	mean and variance of one squared coordinate difference
w, b	the hyperplane's coefficients and intercept
ξ_i	how far example i violates the margin
C	the price of a margin violation
γ	how far one point's influence reaches, in an RBF kernel
a_i	the multiplier on example i's margin constraint (usually written α_i)
ϕ	the map into the higher-dimensional space
K	the kernel, an inner product in that space